

CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION

The Pepperfry E-Commerce Database System is designed to efficiently manage customers, products, categories, brands, suppliers, carts, orders, order details, payments, deliveries, and reviews. Functional dependencies help identify the relationships between attributes in each entity. Redundancy analysis helps determine whether the same data is unnecessarily repeated in multiple records. Data anomaly analysis helps identify possible insertion, update, and deletion anomalies. A properly structured and normalized database reduces unnecessary data duplication and maintains data consistency. The use of primary keys and foreign keys ensures that each entity is uniquely identified and properly connected with other entities. The analysis of functional dependencies and data anomalies helps improve database integrity, reliability, and performance.

1. CUSTOMER TABLE

Functional Dependency

Customer_ID → First_Name, Last_Name, Email, Phone_Number, Address, Registration_Date

Redundancy Analysis

Customer details are stored only once in the Customer table. Other tables such as Orders, Cart, and Review store only **Customer_ID** as a foreign key instead of repeating customer details.

Data Anomaly Analysis

No significant anomalies.

Reason: Customer information is maintained in one place. If a customer's address or phone number changes, it needs to be updated only once. Deleting an order or review does not delete the customer's personal information. Therefore, update and deletion anomalies are avoided.

2. PRODUCT TABLE

Functional Dependency

Product_ID → Product_Name, Category_ID, Brand_ID, Price, Stock, Material, Color

Redundancy Analysis

Product information is stored only in the Product table. Other tables such as Order_Details, Cart, and Review refer to products using **Product_ID** instead of storing the complete product information repeatedly.

Data Anomaly Analysis

No significant anomalies.

Reason: Product details such as price, stock, material, and color are maintained in a single table. Any changes to product information can be made in one place, preventing inconsistent product data.

3. CATEGORY TABLE

Functional Dependency

Category_ID → Category_Name, Description

Redundancy Analysis

Each category is stored only once in the Category table. Products use **Category_ID** as a foreign key instead of repeating the category name and description.

Data Anomaly Analysis

No significant anomalies.

Reason: Category information is stored separately from product information. If a category name or description changes, it can be updated in one place without modifying every product record.

4. BRAND TABLE

Functional Dependency

Brand_ID → Brand_Name, Country

Redundancy Analysis

Brand details are stored only once in the Brand table. Products reference the brand using **Brand_ID**, preventing repeated storage of brand names and country information.

Data Anomaly Analysis

No significant anomalies.

Reason: A brand can be associated with multiple products, but the brand information is stored only once. Therefore, updating or deleting a product does not affect the brand's main information.

5. SUPPLIER TABLE

Functional Dependency

Supplier_ID → Supplier_Name, Contact_Number, Email, Address

Redundancy Analysis

Supplier information is maintained only once in the Supplier table. Products reference the supplier through **Supplier_ID** instead of repeating supplier details.

Data Anomaly Analysis

No significant anomalies.

Reason: Supplier details are stored independently. If the supplier's contact number, email, or address changes, the information can be updated in a single location. This prevents update anomalies.

6. ORDERS TABLE

Functional Dependency

Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status, Payment_ID

Redundancy Analysis

The Orders table stores only order-related information. Customer details are accessed through **Customer_ID**, and payment information is connected using **Payment_ID**.

Data Anomaly Analysis

No significant anomalies.

Reason: Customer and payment details are not repeatedly stored in the Orders table. Updating an order status does not require changing customer or payment information. Deleting an order also does not directly delete the customer record.

7. ORDER_DETAILS TABLE

Functional Dependency

Order_Detail_ID → Order_ID, Product_ID, Quantity, Unit_Price

Redundancy Analysis

The Order_Details table acts as a linking table between Orders and Products. It stores only the quantity and unit price related to a particular order item instead of repeating complete product details.

Data Anomaly Analysis

No significant anomalies.

Reason: This table efficiently manages the relationship between orders and products. A single order can contain multiple products, and the same product can appear in different orders. Product details are not duplicated in every order record.

8. CART TABLE

Functional Dependency

Cart_ID → Customer_ID, Product_ID, Quantity

Redundancy Analysis

The Cart table stores only the customer, product, and quantity information required for cart management. Complete customer and product details are accessed through their respective foreign keys.

Data Anomaly Analysis

Possible deletion anomaly.

Reason: If a cart record is deleted, the relationship between the customer and the product in the cart is removed. However, the actual customer and product information is not deleted because it is stored separately in the Customer and Product tables. Therefore, the anomaly is limited to cart information and does not affect the main entity data.

9. PAYMENT TABLE

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date

Redundancy Analysis

Payment details are stored separately from the Orders table. The Payment table stores payment-specific information and is linked to an order using **Order_ID**.

Data Anomaly Analysis

No significant anomalies.

Reason: Payment details are maintained independently. Updating the payment status does not require modifying customer or product information. Payment information can also be deleted without deleting the related customer or product data.

10. DELIVERY TABLE

Functional Dependency

Delivery_ID → Order_ID, Delivery_Address, Delivery_Date, Delivery_Status

Redundancy Analysis

Delivery information is stored separately from the Orders table. The Delivery table contains delivery-specific attributes and refers to the related order using **Order_ID**.

Data Anomaly Analysis

No significant anomalies.

Reason: Delivery status and delivery date can be updated independently without changing order or customer information. This separation prevents unnecessary duplication of delivery details.

11. REVIEW TABLE

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

Review information is stored separately. Customer and product details are not repeated in every review. The Review table uses **Customer_ID** and **Product_ID** as foreign keys.

Data Anomaly Analysis

No significant anomalies.

Reason: Reviews are maintained independently from customer and product details. Updating or deleting a review does not affect the customer or product records. Therefore, update and deletion anomalies are avoided.

OVERALL DATA ANOMALY ANALYSIS

Table	Data Anomaly	Reason
Customer	No significant anomalies	Customer details are stored only once and referenced using Customer_ID.
Product	No significant anomalies	Product information is maintained in one table, avoiding repeated product data.
Category	No significant anomalies	Category details are stored separately and referenced using Category_ID.
Brand	No significant anomalies	Brand information is stored once and linked to products using Brand_ID.
Supplier	No significant anomalies	Supplier details are stored independently, preventing repeated contact information.
Orders	No significant anomalies	The table stores only order-related data and uses foreign keys for related entities.
Order_Details	No significant anomalies	It efficiently connects orders and products without repeating complete product details.
Cart	Possible deletion anomaly	Deleting a cart record removes the cart relationship, although the main customer and product data remain safe.
Payment	No significant anomalies	Payment information is maintained separately from customer and order details.
Delivery	No significant anomalies	Delivery updates can be performed independently of order and customer data.
Review	No significant anomalies	Review data is stored independently and does not affect customer or product records.

CONCLUSION

The Peppery E-Commerce Database System is designed using appropriate functional dependencies and a normalized database structure. Each table stores data related to a specific entity, thereby reducing unnecessary redundancy. Primary keys uniquely identify records, while foreign keys establish relationships between different entities.

Most tables do not have significant insertion, update, or deletion anomalies because the database separates customer, product, category, brand, supplier, order, payment, delivery, and review information into independent tables. The Cart table may have a limited deletion

anomaly because deleting a cart record removes the customer-product cart relationship. However, the main customer and product data remain unaffected because they are stored separately.

Overall, the database design improves data consistency, reduces redundancy, prevents major data anomalies, and supports efficient management of the Pepperfry E-Commerce System